

Table of Contents

1. Multi-Tenancy Models	1
1.1. DEDICATED_REALM : One Tenant per Database	1
1.2. POOLED_REALM : Many Tenants in One Database	1
1.3. Freemium and Trial Tenants	2
1.4. Tenant Onboarding and Access Invites	2
1.4.1. 1. Tenant provisioning: create the tenant itself	2
1.4.2. 2. Tenant decommissioning: archive before topology-specific cleanup	5
1.4.3. 2. Tenant onboarding workflow: define the journey after invite/registration	6
1.4.4. 3. Registration requests: capture onboarding data for approval	7
1.4.5. 4. Access invites: grant a user into an existing tenant	8
1.4.6. Invite extension points	10
1.4.7. How these pieces fit together	10
1.5. Choosing a Tenancy Model: Three Options	11
1.6. The GitHub-Like Identity Model, Mapped	11
1.7. User-Definable Facet Segmentation & Coordinate SPI in Multi-Tenant Environments	12
1.7.1. Physical Realm Isolation vs. Logical Facet Segmentation	12
1.7.2. Single Server Instance with Per-Tenant Heterogeneous Policies	13
1.7.3. Configuring User-Definable Facets	13
1.7.4. Coordinate Resolution Precedence and Defaults	14
1.7.5. Multi-Hop Traversal Isolation	15

Chapter 1. Multi-Tenancy Models

Quantum supports multiple multi-tenant models for MongoDB deployments:

Every persisted `Realm` has a `deploymentType`:

- `DEDICATED` is the default. Realm records created before this field existed, and any record whose value is absent or null, resolve to `DEDICATED`.
- `SHARED` must be set explicitly before the realm can accept pooled tenant admissions.

The system-management realm catalog exposes this value so operators can see the database topology directly. Tenant lifecycle requests use `DEDICATED_REALM` and `POOLED_REALM`; those request values must agree with the realm's persisted `DEDICATED` or `SHARED` type.

Database migration `1.0.6` backfills existing system-realm catalog records. It updates only Realm documents where `deploymentType` is missing or null, setting them to `DEDICATED`; an explicitly configured `SHARED` value is preserved. Deployments should advance `quantum.database.version` to `1.0.6` and run the normal Quantum migration workflow rather than patching Realm documents individually.

1.1. `DEDICATED_REALM`: One Tenant per Database

- Each tenant is mapped to a dedicated MongoDB database within a cluster.
- Strong isolation at the database level; operational controls via MongoDB roles.
- Pros: Simplified backup/restore per tenant; reduced risk of data bleed.
- Cons: More databases to manage (indexes, connections), higher operational overhead.

How Quantum helps:

- `DataDomain` carries tenant identifiers (e.g., `tenantId`, `ownerId`, `orgRefName`) on each model.
- Repositories can resolve connections/DB selection per tenant, enabling routing to the appropriate database.

1.2. `POOLED_REALM`: Many Tenants in One Database

- Multiple tenants share a single database and collections.
- Isolation is enforced at the application layer using `DataDomain` filters.
- Pros: Fewer databases to manage; efficient index utilization and connection pooling.
- Cons: Strict discipline required to enforce filtering and access rules.

How Quantum helps:

- `DataDomain` is part of every persisted model, enabling programmatic, rule-based filtering.
- `RuleContext` and `DomainContext` inject tenant-aware filters into governed repositories and resources.

- Cross-tenant sharing can be modeled by specific DataDomain fields and RuleContext logic granting read access across tenants on a per-functional-area basis.

In both modes, realm means MongoDB placement and schema lifecycle while DataDomain means row visibility. `POOLED_REALM` is therefore safe only for models and repository paths that participate in governed DataDomain filtering. A shared database does not weaken or replace the policy boundary.

1.3. Freemium and Trial Tenants

- Programmatically create tenants to support self-service onboarding.
- Attach time-bound or capability-bound policies.
- Use scheduled jobs to convert/expire trials.

Quantum patterns:

- Tenant onboarding service creates a DataDomain scope and any default records.
- Policies are encoded in RuleContext checks to allow or restrict actions based on time, plan, or feature flags.

1.4. Tenant Onboarding and Access Invites

Quantum separates realm creation, tenant admission, user access, and the user-facing onboarding journey. Those are related, but they are not the same thing:

- Dedicated provisioning creates the tenant realm, runs realm migrations, and establishes the initial admin user.
- Pooled admission adds a tenant to an existing realm and does not create or initialize a database.
- Tenant onboarding workflow defines the steps a tenant user should complete before access is considered fully activated.
- Access invites bring an individual user into an existing tenant and can attach scoped access metadata.

This split helps teams support both admin-driven setup and self-service onboarding without forcing every tenant into a single path.

1.4.1. 1. Tenant provisioning: create the tenant itself

The admin tenant provisioning API is exposed at:

- `POST /admin/tenants`
- `DELETE /admin/tenants/{realmId}`

Provisioning is intended for `admin` and `system` roles. The same endpoint supports both deployment topologies. Omitting `deploymentTopology` preserves the historical `DEDICATED_REALM` behavior. `POOLED_REALM` admission succeeds only when `placementRealmId` identifies an existing Realm explicitly marked `SHARED`; legacy and untyped realms are treated as `DEDICATED`.

Request fields:

Field	Purpose
<code>tenantDisplayName</code>	Human-readable label for the tenant
<code>tenantEmailDomain</code>	Source for the realm id; dots are converted to dashes
<code>orgRefName</code>	Organization reference used in the tenant <code>DomainContext</code> and <code>DataDomain</code>
<code>accountId</code>	Initial account number for the tenant
<code>adminUserId</code>	User id for the initial tenant admin
<code>adminSubject</code>	Optional stable subject override; defaults to <code>adminUserId</code>
<code>adminPassword</code>	Initial password for the admin user
<code>applicationId</code>	Application that owns the realm placement; may use the configured default
<code>deploymentTopology</code>	<code>DEDICATED_REALM</code> or <code>POOLED_REALM</code> ; when omitted, uses <code>quantum.tenant.provisioning.deployment-topology</code> (default <code>DEDICATED_REALM</code>)
<code>placementRealmId</code>	Existing target realm for <code>POOLED_REALM</code> ; when omitted, uses <code>quantum.tenant.provisioning.pooled-realm</code>
<code>seedArchetypes</code> or <code>archetypes</code>	Optional list of seed archetypes for dedicated realm creation. Pooled admission rejects archetypes until the seed contract can prove all writes are tenant-scoped.

Representative request:

```
{
  "tenantDisplayName": "Acme Logistics",
  "tenantEmailDomain": "acme.example",
  "orgRefName": "acme.example",
  "accountId": "1000000001",
  "adminUserId": "admin@acme.example",
  "adminSubject": "admin@acme.example",
  "adminPassword": "change-me-now",
  "seedArchetypes": ["DemoArchetype"]
}
```

Representative response:

```
{
  "realmId": "acme-example",
  "realmCreated": true,
  "userCreated": true,
  "appliedSeedArchetypes": ["DemoArchetype"],
  "warnings": []
}
```

What provisioning does:

- derives `realmId` from `tenantEmailDomain`
- writes a `Realm` catalog record in the system realm
- builds tenant `DomainContext` and `DataDomain`
- runs migrations in the new tenant realm
- creates or validates the initial admin user
- applies applicable base seed packs
- optionally applies requested seed archetypes
- applies indexes and verifies initialization

For `POOLED_REALM`, provisioning instead:

- derives a distinct tenant id from `tenantEmailDomain`
- resolves `placementRealmId` to an existing `Realm` catalog record
- creates a PARTICIPANT `RealmTenantMembership`
- assigns the tenant's users to the shared realm with the tenant's own `DomainContext` and `DataDomain`
- creates only the requested administrator identity and tenant-scoped realm assignment; it does not create baseline or demo identities
- skips realm migrations, base seed packs, archetypes, and index initialization
- never creates or drops the shared realm/database

The pooled realm must therefore be provisioned and migrated before admitting tenants. Every persisted model reachable by a pooled tenant must participate in governed `DataDomain` filtering for reads and writes. A user currently has one active tenant `DataDomain` assignment per realm; admitting the same identity into a second tenant in that realm fails closed until an explicit tenant-selection claim is available.

Example pooled admission:

```
{
  "tenantDisplayName": "Acme Logistics",
  "tenantEmailDomain": "acme.example",
  "orgRefName": "acme.example",
  "accountId": "1000000001",
  "adminUserId": "admin@acme.example",
  "adminPassword": "change-me-now",
  "deploymentTopology": "POOLED_REALM",
  "placementRealmId": "shared-logistics"
}
```

Provisioning is designed to be largely idempotent. If the same tenant or admin user already exists with matching attributes, the service proceeds and returns warnings instead of blindly recreating

state. If existing state conflicts with the request, the service rejects the operation.

1.4.2. 2. Tenant decommissioning: archive before topology-specific cleanup

Tenant deletion is a governed lifecycle operation, not a direct CRUD delete. The placement topology determines the cleanup boundary:

- **DEDICATED_REALM**: revoke tenant access, create and verify a database archive, then drop the tenant database and retire its realm catalog state.
- **POOLED_REALM**: revoke tenant access, export only rows matching the canonical tenant **DataDomain** identity (**orgRefName**, **accountNum**, **tenantId**, and **dataSegment**), then stamp every matching row with one immutable purge batch reference and one **purgeAfter** timestamp. The shared database and realm are never dropped.

MongoDB TTL cleanup is asynchronous and may remove documents from different collections at different times. Revoking tenant access before expiration is therefore the logical atomic barrier: users cannot observe a partially drained tenant. Before the workflow reports the pooled purge as scheduled, it must reconcile archive and expiration counts for every collection declared tenant-scoped.

Revocation verification includes already-issued tokens or sessions, not only future logins. Suspending a system-plane membership by itself is not a safe barrier for a stateless tenant plane. A deployment must install a revocation provider that invalidates active access (or waits behind an equivalent verified write/read fence) before archive and expiration proceed.

Before a pooled purge is planned, the persisted collection inventory must be reconciled with the collection ownership catalog. Every existing user collection must have exactly one scope:

- **TENANT_SCOPED**: selectable by the canonical tenant identity and eligible for tenant archive and expiration.
- **REALM_SHARED**: shared within the realm and excluded from every individual tenant purge.
- **SYSTEM_GLOBAL**: owned by the system management plane and excluded from tenant purge.

An existing collection without a scope declaration blocks the operation. Declaring a collection as shared or global also forbids tenant-selection and expiration field mappings for that collection.

The default Mongo registry is configured with comma-separated collection names:

```
quantum.tenant-storage.tenant-scoped-  
collections=orders,orderEvents,tenantFiles.files,tenantFiles.chunks  
quantum.tenant-storage.realm-shared-collections=realmConfiguration  
quantum.tenant-storage.system-global-collections=
```

Only MongoDB internal **system.*** namespaces are excluded automatically. GridFS and other blob collections are ordinary user collections for this purpose and must be classified explicitly. Reading the inventory does not create a Morphia datastore, collections, or indexes.

TTL indexes must exist on the configured **purgeAfter** field for every tenant-scoped collection. Realm

bootstrap/migrations should install them; the default Mongo expiration provider also creates the absolute-time TTL index idempotently before it stamps rows. Large collections may be processed in resumable batches, but every batch uses the same purge reference and timestamp. Blob payloads owned by tenant rows are part of both archive and expiration coverage.

`ownerId` is not part of tenant deletion identity. It can narrow policy access, but using it for archive or purge selection can leave tenant-owned data behind. Direct `DELETE /admin/tenants/{realmId}` remains disabled until a governed control-plane workflow supplies archive verification, collection-scope metadata, and purge reconciliation.

Direct tenant deletion is disabled. Decommissioning must use the governed control-plane lifecycle. In particular, pooled tenant removal is a scoped DataDomain operation and must never drop its shared realm/database.

1.4.3. 2. Tenant onboarding workflow: define the journey after invite/registration

Tenant onboarding workflow configuration is exposed at:

- `GET /onboarding/workflow/current`
- `POST /onboarding/workflow/current`

This is tenant-scoped configuration. The active realm is derived from the current security context, so the workflow is saved and read per tenant realm.

The persisted model is `TenantOnboardingWorkflow`, which includes:

- `workflowEnabled`
- `inviteRequired`
- `registrationRequired`
- `surveyRequired`
- `adminApprovalRequired`
- `autoAssignSurveyOnInvite`
- `defaultSurveyRefName`
- `defaultInviteEmailTemplateKey`
- `workflowDefinitionJson`
- `completionMessage`

`defaultInviteEmailTemplateKey` is the email-template key the onboarding flow should use for invite messages. The framework default is `tenant-onboarding-invite`, which can be satisfied by:

- a classpath template under `templates/email/tenant-onboarding-invite/`
- a realm-scoped override in `email_templates`
- a workflow update that points to a different key

By default, the workflow steps are generated in this order:

- `invite`
- `registration`
- `survey`
- `approval`
- `activation`

Each step can be enabled or omitted by toggling the corresponding booleans. When no custom workflow JSON is supplied, the framework generates a default `workflowDefinitionJson` using those flags.

Representative response:

```
{
  "refName": "default-tenant-onboarding",
  "displayName": "Tenant User Onboarding",
  "activeStatus": true,
  "inviteRequired": true,
  "registrationRequired": true,
  "surveyRequired": true,
  "adminApprovalRequired": true,
  "autoAssignSurveyOnInvite": true,
  "defaultSurveyRefName": "tenant-user-onboarding",
  "defaultInviteEmailTemplateKey": "tenant-onboarding-invite",
  "completionMessage": "Your tenant onboarding is complete. You can now sign in and begin using the workspace.",
  "steps": [
    { "key": "invite", "type": "access_invite", "required": true },
    { "key": "registration", "type": "registration_request", "required": true },
    { "key": "survey", "type": "survey", "required": true, "surveyRefName": "tenant-user-onboarding" },
    { "key": "approval", "type": "admin_approval", "required": true },
    { "key": "activation", "type": "account_activation", "required": true }
  ]
}
```

This workflow config answers "what steps should happen?" It does not by itself send invites or create tenant users.

1.4.4. 3. Registration requests: capture onboarding data for approval

The registration-request endpoints are exposed at:

- `POST /onboarding/registrationRequest/create`
- `POST /onboarding/registrationRequest/approve`

This API is the registration step that the onboarding workflow can reference through the

`registration_request` step type. It is useful when a tenant wants users or organizations to submit onboarding information before access is approved.

At a high level:

- `create` persists an `ApplicationRegistration`
- `approve` transitions an existing registration request through the repository approval logic

This is the structured "please review my onboarding submission" side of the journey, whereas access invites are the "you have been invited into this tenant" side.

1.4.5. 4. Access invites: grant a user into an existing tenant

Access invites are exposed at:

- `GET /access/invites`
- `GET /access/invites/{refName}`
- `POST /access/invites`
- `POST /access/invites/{refName}/revoke`
- `POST /access/invites/accept`

The first four require an authenticated caller in the target tenant. Invite acceptance is `@PermitAll` because the recipient may not yet have tenant access.

The persisted `AccessInvite` model includes:

- `email` or `targetUserId`
- `invitedByUserId`
- `scopeRefs`
- `grantedRoles`
- `allowedFunctionalAreas`
- `allowedFunctionalDomains`
- `allowedActions`
- `inviteMessage`
- `expiresAt`
- `acceptedAt`
- `acceptedUserId`
- `status` of `PENDING`, `ACCEPTED`, `REVOKED`, or `EXPIRED`

Representative invite creation request:

```
{  
  "email": "jane.doe@partner.example",  
}
```

```

"scopeRefNames": ["north-america"],
"grantedRoles": ["user"],
"allowedFunctionalAreas": ["integration"],
"allowedFunctionalDomains": ["exchange", "workflow"],
"allowedActions": ["view", "create"],
"expiresInDays": 14,
"inviteMessage": "Join the tenant workspace for partner onboarding."
}

```

Representative creation response:

```

{
  "refName": "invite-1775100000000",
  "email": "jane.doe@partner.example",
  "scopeRefNames": ["north-america"],
  "grantedRoles": ["user"],
  "allowedFunctionalAreas": ["integration"],
  "allowedFunctionalDomains": ["exchange", "workflow"],
  "allowedActions": ["view", "create"],
  "status": "PENDING",
  "inviteToken": "raw-token-returned-at-create-time-only"
}

```

Important invite behavior:

- either `email` or `targetUserId` is required
- only one active pending invite is allowed for the same email or target user
- tokens are stored as a hash; the raw token is only returned at invite creation time
- invites expire automatically based on `expiresAt`
- invites can be revoked before acceptance

Representative accept request:

```

{
  "realm": "acme-example",
  "token": "raw-token-returned-at-create-time-only",
  "email": "jane.doe@partner.example",
  "firstName": "Jane",
  "lastName": "Doe",
  "password": "choose-a-password"
}

```

Representative accept response:

```

{
  "userId": "jane.doe@partner.example",

```

```
"email": "jane.doe@partner.example",
"defaultRealm": "acme-example",
"inviteRefName": "invite-1775100000000",
"grantedScopes": ["north-america"]
}
```

What happens when an invite is accepted:

- the invite token is validated and checked for expiry
- the user identity is resolved from the authenticated user, request payload, invite target user, or invite email
- if the user does not already exist, a new credential is created and a password is required
- if the user already exists, the service ensures the user is authorized for the tenant realm
- a `UserProfile` is created if needed
- the invite status is changed to `ACCEPTED`

1.4.6. Invite extension points

The invite flow is intentionally extensible through two SPIs:

- `AccessInviteProvisioner`
- `AccessInviteNotificationService`

`AccessInviteProvisioner` can:

- validate requested scopes when the invite is created
- perform application-specific provisioning when the invite is accepted

`AccessInviteNotificationService` can:

- deliver the invite token through email, messaging, or another notification mechanism

The framework ships with no-op default beans for both, so applications can opt in to deeper behavior without rewriting the invite flow.

1.4.7. How these pieces fit together

A common pattern looks like this:

1. An admin provisions the tenant with `POST /admin/tenants`.
2. The tenant configures or accepts the default onboarding workflow with `GET/POST /onboarding/workflow/current`.
3. A tenant admin creates an access invite for a user with `POST /access/invites`.
4. The invited user accepts the invite with `POST /access/invites/accept`.
5. If the workflow requires registration, survey, or approval, those steps are completed before the

tenant considers onboarding complete.

This separation gives you flexibility:

- use tenant provisioning for admin-created tenants
- use registration requests for self-service intake and approval
- use access invites for controlled user admission into an existing tenant
- use workflow configuration to decide which of those steps are mandatory in each tenant

1.5. Choosing a Tenancy Model: Three Options

With realm membership (see [design/realm-membership-adr.adoc](#)) the platform supports three deliberate configurations:

Option	Shape	When to use
One tenant, one realm (default)	<code>POST /admin/tenants</code> provisions a realm with a single OWNER membership.	The standard SaaS tenant: full physical isolation, independent lifecycle, own backups/residency. This is what a multi-tenant platform provisions per customer.
Shared collaboration realm	One OWNER org plus N PARTICIPANT memberships (<code>RealmTenantMembership.membershipRole</code>). All parties' data lives in one realm; visibility is shaped per org by DataDomain stamps plus relationship policies (<code>hasEdge</code>) and field-level exclusions.	Multi-party operations against shared entities — e.g. a fulfillment realm where carrier, supplier, and customer orgs collaborate on the same sales orders and fulfillment plans, each seeing their own slice.
GitHub-like multi-realm identity	Users hold ONE global identity and N per-realm role assignments (<code>UserRealmRole</code>): ADMIN in realm A, VIEWER in realm B.	Consultants, partners, and platform operators who work across several tenants without duplicate accounts.

1.6. The GitHub-Like Identity Model, Mapped

GitHub concept	Quantum primitive	Notes
Account	Credential (system realm)	One global identity; <code>defaultRealm</code> is the personal/default context.
Organization	Organization + Account entities	Carry DataDomain identity inside realms.

GitHub concept	Quantum primitive	Notes
Org membership + role	<code>UserRealmRole</code> (userId, realmRefName, roles[], sponsoringOrgRefName)	ADMIN in A / VIEWER in B = two documents. <code>status</code> : active/invited/suspended.
Repository	Realm	The unit of placement and lifecycle (own database, migrations, seeds).
Repo transfer / ownership	<code>RealmTenantMembership</code> . <code>membershipRole = owner</code>	Exactly one owner per realm; owner holds lifecycle and billing authority.
Invitation	<code>AccessInvite</code> (/access-invites)	Existing invite flow grants a user into a realm; on acceptance a <code>UserRealmRole</code> records the granted roles.
Token scopes	JWT per-realm role claims	At issuance the IdP resolves <code>RealmMembershipService.rolesForUser(user, realm)</code> ; the tenant plane validates via JWKS and routes by claims — no callback to the control plane per request.

Resolution APIs live in `RealmMembershipService` (quantum-system): `membershipsForOrg`, `membersOfRealm`, `ownerOfRealm`, `realmsForUser`, `rolesForUser`. Embedded deployments resolve locally; remote (tier-2) deployments get the same surface backed by the control-plane client (Phase C).

1.7. User-Definable Facet Segmentation & Coordinate SPI in Multi-Tenant Environments

Traditional multi-tenancy models stop at isolating database collections by a single tenant ID. However, enterprise SaaS platforms often require multidimensional logical segmentation within and across tenants — such as isolating data by geographic region, operating subsidiary, business unit, partner channel, or sensitivity facet.

Quantum delivers this capability through the **Store-Free Coordinate SPI** (`DataDomainResolver`) and **User-Definable Facet Segmentation**, allowing a single server instance to support heterogeneous segmentation policies for different tenants.

1.7.1. Physical Realm Isolation vs. Logical Facet Segmentation

Quantum maintains a strict distinction between physical database placement and logical row-level segmentation:

- **Physical Realm Isolation:** Dictates MongoDB cluster and database routing (`DEDICATED` vs `SHARED / POOLED`). It acts as an inviolate physical admission gate.
- **Logical Facet Segmentation:** Dictates which rows, relationships, and fields within a realm are visible to a given principal based on multidimensional coordinates (e.g. `tenantId`, `orgRefName`, `department`, `region`, `classificationTier`).

1.7.2. Single Server Instance with Per-Tenant Heterogeneous Policies

A single Quantum server instance natively supports distinct, user-defined segmentation policies for different tenants without requiring separate runtime processes or re-deployments:

- **Tenant Isolation:** In dedicated realms, policies reside in the tenant's own database; in pooled realms, policies are isolated by `tenantId`.
- **Custom Dimension Modeling:** Tenant A can segment data by `department` and `complianceTier`, while Tenant B concurrently segments data by `region` and `partnerId`.
- **Dynamic Variable Injection:** Policies automatically bind contextual coordinates into `RuleContext` (`${pTenantId}`, `${orgRefName}`, `${ownerId}`, `${policyFilter}`, and custom facet variables), which are evaluated transparently across both single-collection queries and multi-hop aggregation pipelines.

1.7.3. Configuring User-Definable Facets

Facet definitions and policy rules are declared via `DataDomainComponentBinding` and `DataDomainPolicyEntry`:

Configuring Facet Bindings (`DataDomainComponentBinding`)

```
{
  "tenantId": "tenant-acme",
  "name": "OrderSegmentation",
  "facets": {
    "region": {
      "type": "FIELD",
      "targetPath": "shippingAddress.region"
    },
    "department": {
      "type": "FIELD",
      "targetPath": "departmentCode"
    },
    "classification": {
      "type": "VARIABLE",
      "targetPath": "principal.securityClearance"
    }
  }
}
```

Configuring Policy Constraints (`DataDomainPolicyEntry`)

```
{
  "tenantId": "tenant-acme",
  "targetClass": "com.e2eq.sample.Order",
  "action": "find",
  "filter": "status:!\"ARCHIVED\" && region:${userRegion}",
  "facetFilters": {
    "department": "${userDepartment}",
  }
}
```

```

    "classification": "CONFIDENTIAL"
  }
}

```

1.7.4. Coordinate Resolution Precedence and Defaults

When a request arrives at `QueryGatewayResource` or a governed `MorphiaRepo`, `DataDomainResolver` resolves coordinates using a strict three-tier precedence hierarchy:

```

flowchart TD
    Req["Incoming Query / Traversal Request"] --> T1["Tier 1: Tenant-Specific Policy?<br/>(systemDataDomainPolicies for Class + Action)"]
    T1 -- Yes --> ApplyTenant["Apply Tenant Policy & Bind Custom Facets"]
    T1 -- No --> T2["Tier 2: Global / System Policy Provider?"]
    T2 -- Yes --> ApplyGlobal["Apply Global Policy & Bind Standard Facets"]
    T2 -- No --> T3["Tier 3: Caller PrincipalContext Present?<br/>(tenantId, ownerId, orgRefName)"]
    T3 -- Yes --> ApplyPrincipal["Apply Principal Default Coordinates"]
    T3 -- No --> FailClosed["DataDomainResolution.Unresolvable<br/>(FAIL CLOSED - HTTP 500 / 403)"]

    ApplyTenant --> Execute["Execute Query / Traversal Pipeline"]
    ApplyGlobal --> Execute
    ApplyPrincipal --> Execute

```

Resolution Defaults Table

Resolution Stage	Resolution Source	Default / Fallback Behavior
Tenant Policy Configured	<code>systemDataDomainPolicies</code> collection for tenant.	Employs tenant's custom <code>filter</code> and <code>facetFilters</code> . Variables are injected into <code>RuleContext</code> .
No Tenant Policy Configured	System Global Policy Provider.	Employs platform-wide default data domain rules if registered; otherwise falls back to Tier 3.
Fallback (Zero Policy Records)	Caller's <code>PrincipalContext</code> .	Resolves default coordinates directly from user credentials: <code>pTenantId = principal.tenantId</code> , <code>ownerId = principal.userId</code> , <code>orgRefName = principal.orgRefName</code> .
Missing Security Credentials	None.	Yields <code>DataDomainResolution.Unresolvable</code> . Fails closed immediately with HTTP 500 (<code>SecurityContextMissing</code>) or HTTP 403.

1.7.5. Multi-Hop Traversal Isolation

During multi-hop traversals (e.g. `Customer` → `Order` → `Supplier` via `expand(...)`):

1. `DataDomainResolver.resolveForHop(targetClass, viaPath, context)` is invoked at each relationship boundary.
2. Target entity policies and row-level security constraints are applied to that hop's MongoDB `$lookup` pipeline.
3. Access constraints **accumulate monotonically**: a caller cannot hop across a foreign reference to bypass tenant boundaries or see records they are not authorized to view in the target domain.
4. References pointing to external tenants or unauthorized records are pruned silently, returning `null` or empty arrays to eliminate information leakage.